

LEARNING RESOURCE GUIDE
ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional
Participant Workbook

Change, Release, Deployment, and Configuration Management

1. Learning Objectives

By the end of this lecture, you will be able to:

- Explain the change management lifecycle including the RFC, the Change Advisory Board, and the three change type categories.
- Describe release management's role in packaging, testing, and version-controlling changes before production deployment.
- Compare blue/green, canary, and rolling deployment strategies and identify the security benefit of each.
- Define the Configuration Management Database (CMDB) and explain how baselines and configuration drift relate to cloud security risk.
- Explain how desired state configuration tools enforce compliance continuously rather than through periodic audits.

2. Introduction

A security engineer at a global e-commerce company receives an alert at 11 PM on a Friday: the production payment processing service is returning errors for roughly 30 percent of transactions. The on-call team traces the failure to a database endpoint change that went live two hours earlier. The change was made directly to the production environment by a developer who needed to test a configuration update before the weekend traffic peak. There was no RFC. There was no CAB approval. There was no staging test. There was no rollback plan. The recovery took six hours, affected millions of transactions, and became a compliance finding under PCI-DSS because cardholder data systems had been modified without proper change authorization.

This scenario is not unusual. Post-incident analyses across cloud security breaches and major outages consistently identify unauthorized or poorly governed change as a primary root cause. The disciplines of change management, release management, deployment management, and configuration management exist precisely to prevent this failure mode — and to provide the audit trail that regulators, customers, and internal security teams require as evidence that cloud environments are managed with rigor.

This lecture covers four interconnected operational disciplines that form the backbone of a well-governed cloud environment. While they are distinct processes, they hand off to each other continuously: changes are formally requested, reviewed, and approved; releases package those changes for controlled deployment; deployment strategies manage how

releases reach production with minimum risk; and configuration management ensures the resulting environment remains in its approved state. Mastering the vocabulary and structure of each discipline is essential for the CCSP exam and for cloud security operations practice.

3. Core Concepts Explained

3.1 Change Management

Definition: Change management is the formal process for requesting, evaluating, approving, implementing, and reviewing changes to any IT service component. A change is defined as any addition, modification, or removal of anything that could affect IT services, including infrastructure, configuration, application code, security policies, and network routes.

Business Relevance: In cloud environments, change velocity is high. Infrastructure is provisioned and deprovisioned programmatically, application code is deployed multiple times daily, and security configurations are adjusted in response to threat intelligence. Without a formal change management process, each of these changes accumulates risk. Audit trails are missing, rollback procedures are undefined, and the blast radius of a failed change is unknown until it occurs.

Practitioner Angle: The Request for Change (RFC) is the formal document that initiates the change process. A well-structured RFC captures: what is changing and why, which systems are affected, what testing has been completed, what the expected outcome is, and — critically — what the rollback procedure is if the change fails. The Change Advisory Board (CAB) reviews RFCs and assesses risk, business impact, and scheduling before granting approval. The CAB typically includes representatives from operations, security, application teams, and business stakeholders, ensuring that no single function can approve a high-risk change unilaterally.

Real-World Example: The 2021 Facebook outage, which took Facebook, Instagram, and WhatsApp offline for approximately six hours, was caused by a configuration change to backbone routers that inadvertently severed the company's global network. While Facebook has not published a detailed RFC analysis, the incident demonstrated the catastrophic potential of a change to a shared network component that propagates across all dependent services simultaneously — precisely the risk that a CAB process with formal impact assessment is designed to surface and prevent.

Change Types and Their Security Implications: Normal Changes go through the full RFC and CAB process, appropriate for significant changes with meaningful risk. Standard Changes are pre-approved templates for routine, low-risk changes — applying a known patch to a well-understood system type — allowing operations teams to act quickly without convening a CAB each time. Emergency Changes exist for situations requiring immediate action, such as applying a zero-day patch, making a network change to contain an active intrusion, or reverting a failed production deployment. Emergency Changes use an abbreviated approval path but require mandatory post-implementation review.

Risks: Changes made outside the formal process (unauthorized changes) are the leading governance failure in cloud security incidents. Impact assessment failures — where a change to a shared component's cascading effects are not identified before approval — can cause outages and compliance violations across multiple workloads simultaneously.

3.2 Release Management

Definition: Release management handles the planning, packaging, testing, and scheduling of changes into coherent release units. A release is a collection of changes deployed together as a controlled unit, ensuring dependencies are understood and tested before reaching production.

Business Relevance: Release management introduces the discipline of staged progression — from development to testing to staging to production — before any change reaches the environment that serves real customers or processes regulated data. This staging requirement is frequently mandated by compliance frameworks including PCI-DSS (which requires that change testing be performed in a separate environment) and SOC 2 (which assesses change management controls as part of its availability and processing integrity criteria).

Practitioner Angle: Version control is the technical foundation of release management. Every artifact included in a release — application code, infrastructure-as-code templates (Terraform, CloudFormation), Kubernetes manifests, security policy configurations, and network definitions — should be stored in a version control repository with meaningful commit messages, pull request reviews, code ownership assignments, and release tags. This traceability is indispensable for incident response: when something breaks in production, the version control history provides a precise, auditable record of what changed, when, and who approved it.

Real-World Example: In 2012, Knight Capital Group lost \$440 million in 45 minutes due to a software deployment failure. An eight-year-old code path was accidentally reactivated when a new release was deployed to only seven of eight production servers. The eighth server retained old code that behaved differently from the new version. The incident was entirely preventable with modern release management practices: a complete, verified deployment to all production instances, with deployment validation checks and automated rollback on error detection. Knight Capital subsequently went out of business as a result.

Risks: Incomplete deployments where some instances run old code and others run new code. Missing version tags that make it impossible to identify exactly what is running in production at a given moment. Skipping staging environment testing under time pressure, particularly for 'urgent' but non-emergency changes.

3.3 Deployment Management

Definition: Deployment management governs the mechanics of moving a release from staging to production. In cloud environments, deployment strategies have evolved

significantly beyond simple file uploads, and each strategy offers different tradeoffs between risk, speed, and rollback capability.

Blue/Green Deployments: Two identical production environments run in parallel. Traffic is served from the blue environment (current production). The new version is deployed to the green environment and fully tested. Traffic is then shifted from blue to green instantaneously, typically via a load balancer or DNS change. If problems appear post-shift, rollback requires only redirecting traffic back to blue — an operation that takes seconds. Blue/green is the highest-safety deployment strategy because the rollback path is always available and exercised regularly.

Canary Deployments: A small percentage of traffic — typically 1-10 percent — is routed to the new version before the full rollout. Monitoring metrics are observed for error rates, latency, and business-level signals during the canary period. If the new version performs well, the traffic percentage is gradually increased until the full rollout is complete. If the canary shows degradation, the routing is reversed before the problem affects most users. This strategy is particularly useful for detecting issues that only manifest under real user load or specific data patterns.

Rolling Deployments: The new version is deployed to production instances sequentially, in batches, while the remainder of the fleet continues serving the old version. This maintains full capacity throughout the deployment but limits rollback speed because some instances may need to be reverted after the deployment has progressed.

CI/CD Pipelines: Continuous Integration and Continuous Deployment pipelines automate the build, test, and deploy sequence. In a mature cloud environment, a developer's code merge triggers an automated pipeline that builds the application, runs unit and integration tests, performs security scanning (static analysis, dynamic analysis, software composition analysis for vulnerable open-source dependencies), and deploys to production if all gates pass. The security gates within a CI/CD pipeline are particularly valuable because they enforce consistent scanning on every code change rather than relying on periodic manual reviews that are frequently skipped under schedule pressure.

Risks: Deploying to production without running the CI/CD pipeline (manual hotfixes). Security scanning gates that are configured to warn rather than block, allowing known-vulnerable code to reach production. Canary monitoring that is not configured to watch business-level metrics, catching only technical failures while missing functional regressions.

3.4 Configuration Management

Definition: Configuration management identifies, controls, and tracks the state of configuration items (CIs) throughout their lifecycle. A CI is any managed resource: servers, databases, network devices, application configurations, security policies, and documentation. The Configuration Management Database (CMDB) stores the current and historical state of all CIs and the relationships between them.

Business Relevance: The CMDB answers foundational operational questions: what is running in production right now, what version is it, what does it depend on, and when did it last change? Without an accurate CMDB, incident response is impaired — the team cannot quickly identify which systems are affected by a failure — and vulnerability management is incomplete — the team cannot determine which instances are running a vulnerable software version.

Baselines and Configuration Drift: A baseline is the documented, approved state for a CI. When a CIS Benchmark hardening standard is applied to a cloud server, the resulting configuration is the baseline. Configuration drift occurs when a running configuration deviates from its baseline — an unauthorized open port, a disabled security agent, a modified registry setting, or a new user account not provisioned through the approved process. Drift is a security risk because it represents an unapproved state that may have been introduced by an attacker, by an unauthorized configuration change, or by a software update that changed default settings.

Desired State Configuration: Rather than detecting drift through periodic audits, modern cloud environments use desired state tools that continuously compare running configurations against defined baselines and can automatically remediate drift. AWS Config with Config Rules, Azure Policy, and infrastructure-as-code tools like Terraform, Chef, Puppet, and Ansible all support desired state enforcement. When a configuration drifts from its baseline, these tools alert immediately — or in some cases automatically restore the approved configuration — rather than waiting for a quarterly review to surface the deviation.

Real-World Example: In 2019, Capital One suffered a breach affecting over 100 million customers through a misconfigured Web Application Firewall. The WAF configuration had drifted from its approved baseline — an overly permissive firewall rule allowed the attacker to use a Server Side Request Forgery technique to reach the instance metadata service and retrieve AWS credentials. An effective configuration management program with continuous drift detection would have flagged the permissive WAF rule as a deviation from the approved baseline before the attacker could exploit it.

Risks: CMDB data that is not kept current, leading to 'ghost' configurations — systems that exist in production but are not documented, patched, or monitored. Treating desired state tools as optional additions rather than core configuration governance. Relying on periodic audits rather than continuous monitoring for configuration compliance.

4. Deployment Strategy Comparison

Strategy	Mechanism	Rollback Speed	Production Impact During Deploy	Best For

Blue/Green	Two parallel environments; traffic switched at cutover	Immediate (traffic redirect)	Zero — old version runs unchanged	High-risk releases; instant rollback requirement
Canary	Small traffic % routed to new version; gradual rollout	Fast (reduce canary %)	Minimal — only canary % affected	Changes that need real-traffic validation
Rolling	Sequential batch updates across instances	Slower (must revert updated instances)	Partial — some instances on new version	Large fleets; capacity must be maintained
CI/CD Pipeline	Automated build-test-scan-deploy with gates	Depends on deploy strategy used	Controlled by underlying strategy	All cloud deployments; security gate enforcement

4.1 Change Type Comparison

Change Type	Approval Path	Timing	Typical Example	Post-Impl Review Required?
Normal Change	Full RFC + CAB review	Scheduled	Major infrastructure upgrade; new service deployment	Yes
Standard Change	Pre-approved template; no CAB required	Routine	Applying a known security patch per approved template	Yes (periodic template review)
Emergency Change	Abbreviated emergency approval board	Immediate	Zero-day patch; stopping active intrusion	Yes (mandatory post-implementation review)

5. Real-World Incident

Incident: The 2021 Azure Active Directory outage on March 15 affected customers globally for several hours when a backend configuration change to Azure AD's authentication infrastructure caused authentication failures across Microsoft 365, Azure Portal, and dozens of third-party services that depend on Azure AD for identity. Microsoft's post-incident review

acknowledged that the configuration change had propagated to production before the full impact scope had been assessed. The CAB equivalent review had not identified the dependency chain that connected a single authentication backend configuration to thousands of dependent workloads. The incident is a textbook example of change management's most dangerous failure mode: a change to a shared component with an underestimated blast radius and no pre-tested rollback plan that could be executed quickly enough to prevent widespread impact.

6. Memory Anchor

Think of change management, release management, deployment management, and configuration management as the four stages of launching a commercial aircraft. Change management is the engineering decision process — the review board that approves modifications to the aircraft's systems before any wrench is turned. Release management is the maintenance record and test flight — verifying that the modifications work as designed before carrying passengers. Deployment management is the actual boarding and taxiing process — getting the aircraft into service with minimum disruption to the schedule. And configuration management is the ongoing airworthiness registry — the document that records the aircraft's exact current specification and flags any deviation from its certified configuration as requiring immediate investigation. All four stages are mandatory, none is optional, and skipping any one of them is the kind of shortcut that leads to accident investigations.

7. Exam Focus

The CCSP exam tests vocabulary precision for this topic. Know that the RFC is the formal document initiating a change request. The CAB is the multi-stakeholder review body that approves changes. The three change types are Normal, Standard, and Emergency — not 'major,' 'minor,' and 'urgent.'

For deployment strategies, blue/green is the scenario answer when the question emphasizes instant rollback capability or zero-downtime cutover. Canary is the answer when the question involves gradual traffic shifting and real-traffic monitoring before full rollout. Rolling is the answer when the question involves maintaining continuous service availability across a large fleet during the deployment.

For configuration management, the CMDB is the repository, baselines are the reference states, and configuration drift is the deviation from those baselines. Desired state tools provide continuous enforcement rather than periodic discovery. Know the tools: AWS Config, Azure Policy, Terraform, Chef, Puppet, Ansible.

Common exam distractors: Selecting 'patch management' when a question asks about the formal change process — patch management is a subset of change management, not a synonym. Confusing release management with deployment management — release is packaging and testing; deployment is the act of moving to production. Confusing CMDB

(configuration item repository) with a change log (change history) — they capture different information.

8. Key Terms Glossary

- **Request for Change (RFC):** The formal document initiating the change process, capturing what is changing, why, what testing has been done, and what the rollback plan is.
- **Change Advisory Board (CAB):** The multi-stakeholder review body that evaluates RFCs for risk, impact, and scheduling before granting change approval.
- **Standard Change:** A pre-approved change template for routine, low-risk changes that can be executed without convening a CAB.
- **Emergency Change:** A change requiring immediate action with abbreviated approval, followed by mandatory post-implementation review.
- **Release:** A collection of changes packaged and tested as a coherent unit before deployment to production.
- **Blue/Green Deployment:** A deployment strategy using two parallel environments; traffic is switched instantly, enabling immediate rollback.
- **Canary Deployment:** A deployment strategy routing a small traffic percentage to the new version before full rollout.
- **CI/CD Pipeline:** Automated build-test-scan-deploy process that enforces security gates on every code change.
- **Configuration Item (CI):** Any managed resource tracked in the CMDB: servers, databases, network devices, security policies, documentation.
- **CMDB:** Configuration Management Database — the repository storing current and historical CI states and their relationships.
- **Baseline:** The documented, approved configuration state for a CI; the reference point for compliance.
- **Configuration Drift:** Any deviation of a running configuration from its approved baseline — a security risk requiring detection and remediation.
- **Desired State Configuration:** Tools (AWS Config, Azure Policy, Terraform, Ansible) that continuously compare running states to baselines and can auto-remediate drift.

9. Quick Revision Checklist

- ☐ I can describe the RFC contents and explain why a rollback plan is a mandatory RFC element.
- ☐ I can name the three change types (Normal, Standard, Emergency) and describe the approval path for each.
- ☐ I can explain what the Change Advisory Board reviews and why multiple functions are represented on it.
- ☐ I can explain what a release is and why staging environment testing is a compliance requirement in frameworks like PCI-DSS.

- ☐ I can compare blue/green, canary, and rolling deployment strategies by mechanism, rollback speed, and appropriate use case.
- ☐ I can describe what a CI/CD pipeline is and name three types of security scanning that should be gates in the pipeline.
- ☐ I can define the CMDB and explain what questions it answers for incident response and vulnerability management.
- ☐ I can define a baseline and explain configuration drift as a security risk.
- ☐ I can name four desired state configuration tools and explain how they differ from periodic audit approaches.
- ☐ I know the Capital One 2019 breach as a configuration management failure example.

10. Practice Questions

Question 1 (Scenario)

A cloud security engineer discovers that a production database server's security group has an inbound rule allowing port 22 (SSH) access from 0.0.0.0/0 — the entire internet — instead of only from the approved bastion host IP range. There is no record of this change in the change management system, and the deviation was not present in last week's configuration audit. The engineer needs to determine what type of governance failure this represents. Which is the most accurate characterization?

A. A release management failure, because the change was not included in a tested release package.

Explanation: Incorrect. Release management failures involve changes that were not staged or tested. This scenario describes a configuration that has drifted from its approved baseline, which is a configuration management failure, regardless of how the change was introduced.

B. A configuration management failure — specifically, configuration drift from an approved baseline.

Explanation: Correct. The security group rule deviates from the approved baseline configuration. There is no change record, meaning either the change bypassed the change management process (also a change management failure) or it was made by an unauthorized actor. The specific configuration management concept is configuration drift.

C. A deployment management failure, because the change was not deployed using a canary strategy.

Explanation: Incorrect. Deployment strategies govern how releases move from staging to production. This deviation is not a deployment strategy issue; it is a configuration state deviation from baseline.

D. A normal change that was approved and implemented correctly.

Explanation: Incorrect. The absence of a change record and the deviation from the baseline indicate this was an unauthorized change, not an approved normal change.

Question 2 (Scenario)

An e-commerce company is preparing to deploy a new checkout service update that has failed twice in production in the past six months — each time requiring a two-hour rollback procedure during which revenue was lost. The engineering team wants a deployment strategy that allows the new version to serve a small percentage of live traffic first, with the ability to expand or fully revert based on real-time error rate monitoring. Which deployment strategy should they adopt?

A. Blue/Green deployment

Explanation: Incorrect. Blue/green involves a complete cutover of traffic from one environment to another. It offers fast rollback by switching back to the original environment, but it does not natively support graduated traffic routing for pre-release validation with a subset of users.

B. Canary deployment

Explanation: Correct. Canary deployment routes a small percentage of real traffic to the new version while the majority of users continue on the current version. This allows the team to observe error rates and performance under real load before committing to a full rollout, with the ability to revert by reducing the canary percentage.

C. Rolling deployment

Explanation: Incorrect. Rolling deployment updates instances sequentially across the fleet. It maintains some old-version capacity during the deployment but does not support graduated traffic percentage control or easy single-percentage rollback before the full deployment completes.

D. Emergency change deployment

Explanation: Incorrect. Emergency change is a change governance category for situations requiring immediate action, not a deployment strategy. It does not define the technical mechanics of how code is deployed to production.

Question 3 (Scenario)

A compliance team conducting a PCI-DSS assessment asks the cloud security team to demonstrate that changes to the cardholder data environment are tested in a separate environment before being promoted to production, and that the change record includes documentation of the rollback procedure. The security team must point to which cloud operations discipline that satisfies both requirements?

A. Configuration management — specifically, CMDB records showing CI state before and after the change.

Explanation: Incorrect. While CMDB records document CI states, the requirement for staging environment testing and rollback documentation in a change record is addressed by release management and change management together, not the CMDB specifically.

B. Change management (for rollback documentation in the RFC) and release management (for staging environment testing).

Explanation: Correct. The rollback procedure requirement is satisfied by the RFC, which must document the rollback plan as part of the change request. The staging environment testing requirement is satisfied by release management, which mandates that changes be tested in non-production environments before release. Together these two disciplines address the auditor's requirements.

C. Deployment management — specifically, blue/green deployment to ensure instant rollback.

Explanation: Incorrect. Blue/green deployment provides instant rollback capability at the infrastructure level, but it does not itself document the rollback procedure as part of a change record, nor does it require staging environment testing as a formal step.

D. CI/CD pipeline automation, which eliminates the need for manual RFC documentation.

Explanation: Incorrect. CI/CD pipelines automate the technical build-test-deploy process and can enforce testing gates, but they do not replace the governance requirement for RFC documentation of rollback plans. Automation and governance are complementary, not mutually exclusive.

Question 4

A cloud operations team wants to apply routine security patches to a fleet of Linux servers every Tuesday. These patches have been pre-tested on a reference image, are applied to the same server type each week, and follow a documented procedure. What type of change most accurately describes this activity?

A. Normal change requiring full CAB review

Explanation: Incorrect. Weekly routine patching of a pre-tested, well-understood system type meets the definition of a Standard Change — a pre-approved template for a recurring, low-risk activity — rather than a unique change requiring full CAB deliberation.

B. Emergency change requiring immediate approval

Explanation: Incorrect. An Emergency Change is reserved for situations requiring immediate action to prevent or respond to a service failure or security incident. Routine weekly patching is neither urgent nor emergent.

C. Standard change — pre-approved template for routine, low-risk activity

Explanation: Correct. Standard Changes are pre-approved change templates for recurring, low-risk activities where the procedure, systems, and expected outcome are well understood. Weekly security patching of a defined server type following a documented process fits this category exactly.

D. Configuration drift remediation, not a change

Explanation: Incorrect. Configuration drift remediation addresses deviations from the approved baseline. Planned patching is a planned change, not a drift remediation activity, even though the result is an updated configuration.

Question 5

What is the primary purpose of the Change Advisory Board (CAB) in a cloud change management process?

A. To develop the technical design for proposed changes before implementation.

Explanation: Incorrect. The CAB does not develop technical designs. The engineering or architecture team designs the change. The CAB evaluates the resulting RFC for risk, impact, dependencies, and scheduling.

B. To provide multi-stakeholder review of RFCs for risk, business impact, and scheduling before approval.

Explanation: Correct. The CAB brings together representatives from operations, security, application teams, and business stakeholders to assess the RFC. This multi-functional review ensures that security and business impacts are considered before a change is approved.

C. To implement approved changes directly in the production environment.

Explanation: Incorrect. The CAB does not implement changes. It reviews and approves change requests. Implementation is carried out by the engineering or operations team following approval.

D. To maintain the CMDB by recording configuration changes after implementation.

Explanation: Incorrect. The CMDB is maintained as part of configuration management. The CAB's role is pre-implementation review, not post-implementation configuration recording.

Question 6

In a CI/CD pipeline, software composition analysis (SCA) serves which security function?

A. Identifying SQL injection vulnerabilities in application code before deployment.

Explanation: Incorrect. SQL injection vulnerabilities in application source code are identified by static application security testing (SAST) tools. SCA focuses on the open-source dependencies the application uses.

B. Identifying known vulnerabilities in open-source and third-party dependencies included in the application.

Explanation: Correct. Software Composition Analysis scans the application's dependency manifest to identify open-source libraries and components with known vulnerabilities (CVEs). It ensures that vulnerable third-party components are identified and blocked before they reach production.

C. Running the application under simulated attack traffic to discover runtime vulnerabilities.

Explanation: Incorrect. This describes Dynamic Application Security Testing (DAST). SCA does not run the application; it analyzes the dependency inventory against vulnerability databases.

D. Verifying that the build artifact matches the source code in version control.

Explanation: Incorrect. Verifying build provenance and artifact integrity is achieved through code signing and build system controls, not SCA. SCA is focused on identifying vulnerable dependencies.

Question 7

A cloud security team discovers that 15 percent of their EC2 instances have configuration settings that differ from the approved CIS Benchmark baseline. The deviations were not recorded in any change request. What is this condition called, and what tool category would detect and potentially remediate it automatically?

A. Baseline fragmentation; remediated by load balancers

Explanation: Incorrect. Load balancers route traffic between instances; they do not detect or remediate configuration deviations from security baselines.

B. Configuration drift; remediated by desired state configuration tools such as AWS Config or Azure Policy

Explanation: Correct. When running configurations deviate from their approved baselines without a change record, this is configuration drift. Desired state configuration tools — AWS Config with Config Rules, Azure Policy — continuously compare running configurations to defined baselines and can alert on or automatically remediate drift.

C. Release regression; remediated by reverting to the previous release version

Explanation: Incorrect. Release regression occurs when a software release introduces a functional problem. Configuration drift from a security baseline is not a release regression; it is a configuration management failure.

D. Standard change backlog; remediated by the CAB approving all pending changes

Explanation: Incorrect. Configuration drift is not a pending change backlog. It is an unapproved deviation from the approved configuration state.

Question 8

Which deployment strategy provides the fastest rollback capability in the event that a production deployment causes service degradation?

A. Rolling deployment

Explanation: Incorrect. Rolling deployment updates instances sequentially. Rollback requires reverting the updated instances, which takes time proportional to the number of instances already updated.

B. Canary deployment

Explanation: Incorrect. Canary deployment supports fast rollback by reducing the canary traffic percentage, but it does not match blue/green's ability to revert the entire traffic load in a single operation.

C. Blue/green deployment

Explanation: Correct. Blue/green maintains the old version environment in parallel throughout the deployment. Rollback requires only redirecting traffic from the new (green) environment back to the old (blue) environment — typically seconds via a load balancer or DNS change — making it the fastest rollback strategy.

D. Emergency change deployment

Explanation: Incorrect. Emergency change is a change governance category, not a technical deployment strategy. It does not define rollback mechanisms.

Question 9

What is the primary purpose of the Configuration Management Database (CMDB) in cloud security operations?

A. To store application source code and track code changes between releases.

Explanation: Incorrect. Source code and code change tracking are the purpose of version control systems (Git, SVN). The CMDB tracks the state of configuration items — infrastructure, services, and their relationships — not source code.

B. To record and track the current and historical state of configuration items and their relationships.

Explanation: Correct. The CMDB is the authoritative repository of CI state. It answers what is running, what version it is, what it depends on, and when it last changed. This information is essential for incident response, vulnerability management, and compliance auditing.

C. To generate automated alerts when cloud resources exceed capacity thresholds.

Explanation: Incorrect. Capacity alerting is performed by monitoring tools (CloudWatch, Azure Monitor). The CMDB's purpose is configuration state tracking, not performance monitoring.

D. To store approved change templates for Standard Changes.

Explanation: Incorrect. Approved Standard Change templates are stored in the change management system. While related, these are distinct from the CMDB, which tracks CI states rather than change procedures.

Question 10

A company's compliance team notes that an AWS Config Rule is flagging multiple production buckets as non-compliant because they have public access enabled, which violates the approved security baseline. Which change management process is triggered by this finding?

A. A new RFC should be raised to address the drift as a formal change — either restoring the baseline or, if business need exists for public access, updating the baseline through the change process.

Explanation: Correct. When desired state tools identify configuration drift, the appropriate response is to raise an RFC. The RFC either captures the remediation (restoring the approved state) or — if a legitimate business reason exists for the deviation — initiates the process to formally update the approved baseline. Config drift does not self-resolve; it requires a formal change record.

B. An Emergency Change should be raised immediately regardless of whether the deviation represents an active security incident.

Explanation: Incorrect. Emergency Changes are appropriate when immediate action is required to prevent or respond to a live security incident. Configuration drift discovered during a compliance scan may warrant a normal or standard change rather than an emergency change, depending on the risk assessment.

C. The finding can be dismissed if the buckets are not classified as high-impact systems.

Explanation: Incorrect. Configuration drift from a security baseline should be addressed regardless of the system's impact classification. The approved baseline applies to all systems within its scope; dismissing drift based on impact classification creates a permissive exception that undermines the configuration management program.

D. The CMDB should be updated to reflect the current state as the new approved baseline.

Explanation: Incorrect. The CMDB records the current state to identify drift; it does not determine what the approved baseline should be. Updating the CMDB to reflect an unapproved state as the baseline would be a governance failure, not a resolution.